# Nonlinear gadgetization

The product-share encoding, described from scratch

2026-07-24

*Written for a reader who knows the linear (XOR-pair) gadgetizer and nothing else about this line of work. Covers the design of 2026-07-23 and the revisions of 2026-07-24 as one piece. Option reference in §9.*

## 1 Where linear gadgetization leaks

The gadgetizer takes a small circuit $C$ and produces a much larger equivalent $G$ that computes the same function on its low wires. Every logical value of $C$ is **secret-shared** across the wires of $G$, and $G$'s gates manipulate the shares. The security goal is that $G$ reveal nothing about $C$ beyond its input/output behaviour—in particular that an adversary looking at a point of $G$ cannot tell **where in $C$'s computation** that point sits.

In the linear gadgetizer a logical value $v$ lives on two carrier wires as $v = c_0 \oplus c_1$. This is the whole problem. The decode is linear, so **at every instant, every logical value of $C$ is a degree-1 (affine) function of $G$'s wires.** An adversary who fits affine functions can therefore read $C$'s intermediate state out of $G$ at any point—no matter how algebraically complex that state is as a function of the input.

`hmap_affine` measures this directly. For each prefix $i$ of $C$ and prefix $j$ of $G$, it fits the best GF(2) affine reconstruction of $C$'s state after $i$ gates from $G$'s wire values after $j$ gates, and reports held-out error $H$ (0 = perfectly recoverable, 0.5 = hidden). Under linear sharing the aligned cells light up and a low-$H$ **diagonal** runs across the map. That diagonal is a picture of $C$'s computational progress inside $G$.

Read these maps by the **ridge**, never by the mean (which saturates near 0.5): *depth* is the per-row prominence of the low-$H$ valley above its row background, $\rho$ is the Spearman correlation of row index against ridge location ("is it a monotone diagonal"), and *perm-z* scores that against a shuffled null.

**The diagonal survived everything.** Local mixing (ssg, fmix), generation targeting to generation 100+, four million mixing moves, growth phases: all of them shallow the valley somewhat but leave $\rho = 1$. The reason is structural—mixing re-randomizes *which* wires carry the sharing, but the value stays affine in the new wires, so an affine adversary just re-fits. Masking values *between* their uses does not help either: the ridge lives **at** the use points, and any scheme that un-masks an operand so an ordinary gate can read it re-exposes exactly there. (An earlier "deferred mask" design failed on precisely this.)

The leak is not a mixing deficiency. It is the linear decode.

## 2 The fix: a balanced nonlinear decode

Give each logical value, on top of its XOR pair, a set of permanent **multiplicative mask terms**:

$$v_i \;=\; w_u \oplus w_{u'} \;\oplus\; \bigoplus_j \prod_l \big(w_{p_{jl}} \oplus a_{jl}\big) \;\oplus\; c_i$$

Each $\prod_l (w \oplus a)$ is a conjunction of literals over a dedicated band of wires; $c_i$ is a bookkeeping constant held in a ledger, never on a wire. Now the decode is nonlinear, and an affine adversary cannot express it.

**Why the XOR pair is still there.** A pure product decode is un-gateable, and the reason is a counting obstruction worth stating: $(w_p \oplus a)(w_q \oplus b)$ partitions its representations into classes of size 3 and 1, and no reversible gadget can conditionally flip between classes of *unequal* size—a bijection cannot map a 3-element class onto a 1-element class. This holds for every fixed $(a, b)$, with any ancillae, garbage or re-encoding. The linear part re-symmetrizes the classes; it is structural, not a convenience.

## 3 The frozen source band

The mask sources live on a band of extra wires appended after the $2n$ carriers. Each band wire is filled at the input port and then **never written again**. Because the sources are frozen, every registered mask term is **time-invariant**—unaffected by all re-randomization and all gate traffic—so a mask is one gate, not a maintained data structure, and nothing is ever flushed except the final unshare.

**The fill must not be linear.** The obvious fill, a random XOR of data wires, makes every band wire an *affine invariant of the entire circuit*, held from the input port to the output port. That is exactly the class of relation which **collapses the preimage SAT search**: an attacker who learns such invariants from forward runs and injects them turned $n$=32 instances from "$>$2 hours, timeout" into "$\sim$10 minutes". A linear band fill manufactures them for free.

The fill is therefore

$$band_j \;=\; junk \;\oplus\; x_{\text{pivot}} \;\oplus\; (\text{small linear part}) \;\oplus\; \bigoplus^{M} (\text{two-source products})$$

with each product's two sources drawn from the data wires **and from already-filled band wires**. The cascade is the point: multiplying two already-balanced band bits keeps the firing rate near 1/4 while the input degree multiplies up the band, so high degree in $x$ is reached with nothing wider than a two-control gate. (A flat high-degree monomial would fire on only a $2^{-d}$ fraction of inputs and behave statistically like its linear part.)

Balance is exact, which the mask argument needs: the fresh pivot is excluded from the rest of that wire's transitive support, so the fill is balanced over uniform $x$ for any nonlinear part.

The same fill is emitted again after the output bookend (**mirror** $F'$), so the band is not anchored only at one port.

## 4 The share-native fold—the part that actually matters

An encoding is only as good as the gate that operates on it. If a gate reconstructs its operands to compute with them, the ridge comes straight back at the use point.

So the gadget never reconstructs. A source gate $v_t \oplus= f(\text{controls})$ is applied by **expanding $f$'s ANF over the operands' full decodes**. Each control value contributes a set of atoms—its two carrier literals, its mask terms, and a constant when the ledger constant or a negative control polarity applies—and every cross-product of one atom per control becomes **one conjunction fragment** written into a randomly chosen carrier of the target.

The property that matters: a value's own mask term appears only *inside* a fragment's firing condition, as control literals. **It is never materialized on a wire.** No fragment computes a value's two-carrier XOR, and no wire is written except the target's carriers. Operands stay masked straight through the gate. Constants and the gate's complement go to the ledger constant, so the encoding emits no bare $X$ anywhere.

Fragments are emitted in randomized order. Without this, each source gate produced its $(k+2)^2$ fragments consecutively, in deterministic order, all aimed at one value's two carriers—a **progress clock readable statically**, with no execution required.

## 5 Degree and count do different jobs

Two independent levers, and conflating them wastes gates.

**Degree hides algebraically.** A degree-$d$ mask term is not in the degree-$< d$ GF(2) span, so a *single* one forces $H = 0.5$ against **every** reconstruction adversary of degree $< d$, regardless of how many terms there are.

**Count hides statistically.** Against a best-approximation adversary, $k$ stacked terms pile up: the best affine readout error tends to $1/2$ as $1/2 - 2^{-(k+1)}$.

This yields **towers**: a degree-$d$ mask kills every adversary below degree $d$, and the diagonal returns at degree exactly $d$. The wall moves up one level per $+1$ of mask degree; it does not vanish. Verified with real degree-2 and degree-3 adversaries, each capability-controlled.

A degree-$d$ term is also a **sparser** perturbation (it flips the value only $2^{-d}$ of the time), so wider masks are individually *weaker* statistically even as they are stronger algebraically. Hence a **mixed** design: a few low-degree terms for statistical margin plus tower terms for algebraic hiding. The production plan is $[2, 3, 3]$—one degree-2 base term and two degree-3 tower terms. Two degree-3 terms are what kill the degree-2 diagonal; one leaves a boundary residual.

Raising the tower is nearly free in gate count (fragment *count* is set by the term count and the source gate's arity, not by mask degree—widening a mask only adds literals to existing fragments). What it costs is fragment **width**.

## 6 Gate width, and a corrected assumption

Folding a degree-$d$ mask through a $w$-operand gate yields fragments of width up to $d \cdot w$, so the production gadget contains gates of width up to 6. Two consequences were expected to be bad, and exactly one of them is.

**Real:** with `--db-ctrl-cap 2`, any gate with $\geq 3$ controls is ineligible for frozen-DB re-encoding. About 62% of the gadget is in that class. This is why generation targeting cannot report progress here (§8)—and, in the other direction, why the encoding **resists DB-based simplification**, which is a property one wants of an obfuscation.

**Not real:** the assumption that wide gates are immobile fossils. A per-width commutation-leeway census of the real $n$=128 gadget says the opposite—the width-5 and width-6 gates are the **most mobile gates in the circuit** (mean float-box 2779 and 2778, versus 694 for the g57s). A gate with many literals has more chances to hold an *opposite* literal against a neighbour, which exempts the pair from colliding, so it crosses more freely.

A mode exists that removes wide gates entirely (`--prod-max-width`, §9): it ladders every emission down to two controls using a Barenco double sweep over **dirty borrowed carriers**—arbitrary-valued wires, each visited an even number of times so its dirty content cancels between readings. It costs no extra wires and preserves correctness under arbitrary junk. But it costs **6.4**× the gates and *reduces* mobility to mean 259, so it is off by default. Its one live justification is DB re-encodability, and whether that is worth 6.4× is not yet measured.

## 7 What it achieves

All heatmaps read by the ridge measure with verified references, on a real $n$=128 sliced sandwich (source $C = 3000$ g57 gates).

| gadget | adversary | depth | depthMed | $\rho$ | meanH |
|---|---|---|---|---|---|
| linear (plain), 85,169 g / 512 w | degree 1 | 0.346 | 0.356 | **1.000** | 0.475 |
| $[2,3,3]$, 281,068 g / 568 w | degree 1 | 0.044 | **0.000** | 0.015 | 0.499 |
| $[2,3,3]$, 281,068 g / 568 w | degree 2 | 0.043 | **0.000** | −0.007 | 0.498 |

The median original-circuit prefix has **zero** ridge prominence and the reconstruction is uncorrelated with progress. The nulls are capability-controlled: a design whose masks are only degree 2 *is* recovered by the same degree-2 adversary at $\rho = 1.00$, so these are genuine hiding, not a weak measurement.

Earlier work carried the whole pipeline through—gadget → a DB-mixing phase → threefold growth—and the diagonal stayed dead **identically** at every stage. Mixing neither helps nor hurts, because the leak is gone before mixing begins.

**Cost.** The encoding multiplies the plain gadget by ∼3.3× in gates and adds a band of extra wires (56 at $n$=128, versus 512 carriers). Per source gate the fold emits about $(k_{\text{total}} + 2)^2$ fragments, so the term count is a quadratic lever and everything else is second order.

## 8 Reading a mixing run on this material

Generations only advance under DB re-encoding, and ∼62% of a product-share gadget is width $\geq 3$, hence cap-ineligible: those gates sit at generation 0 forever. `G=` is therefore measured over the **targetable** population—cap-eligible and not written off—which is exactly the set the DB channel can move. `Gall=` reports the older all-gates percentile beside it, and on this material that one is **structurally pinned at 0**: it is not a progress number here, and no amount of mixing moves it.

So "run to generation $N$" means $N$ over the targetable material. `--gen-target N` drives it, `--gen-stop-frac` stops on lag/tgtbl, and `--gen-giveup` retires eligible gates the DB genuinely cannot reach. Read `G=`, `lag=` and `tgtbl=`; `wlag=` is the wide population, `Gall=` the legacy figure.

(Before the fix in `512ce31c` both `G=` and the dose stop counted all gates, so on this material the stop could never fire and phase A would burn its whole move budget after the dose was complete—bound

it with `--moves` if running an older build.)

Also required: `--no-local-verify`. The exhaustive per-rewrite check caps support at 24 wires and **panics** on two wide gates. The whole-circuit `--verify-every` check still runs.

# 9 Options

On `sss --cnot --gadgetize`; `gen_sandwich_gadget` reads the same settings from the upper-case environment variables. With both counts at 0 the encoding is off and the gadget is bit-identical to the linear build.

| flag | env var | default | meaning |
|---|---|---|---|
| `--prod-k` | `PROD_K` | 0 | base mask terms per value (statistical) |
| `--prod-deg` | `PROD_DEG` | 2 | literals per base term, min 2 (algebraic) |
| `--prod-k-hi` | `PROD_K_HI` | 0 | tower terms per value |
| `--prod-deg-hi` | `PROD_DEG_HI` | 3 | tower degree; two deg-3 kill the deg-2 diagonal |
| `--prod-band` | `PROD_BAND` | 0 (auto) | source-band width in extra wires |
| `--prod-rsrc` | `PROD_RSRC` | 1 | mask re-source moves per inter-SG gap |
| `--prod-max-width` | `PROD_MAX_WIDTH` | 0 (off) | ladder to this width; `2` = all DB-eligible. $\sim$6.4$\times$ gates |
| `--prod-fill-nl` | `PROD_FILL_NL` | 0 (linear) | nonlinear cascaded band fill. Effectively free |

Mutually exclusive with the deferred-mask `--mask-cov` (RG4). Auto band width is $\approx \max(\sqrt{4nk_{\text{total}}},\ 6,\ \deg_{\max}+3)$

**Production:**

```
sss --cnot --gadgetize \
    --prod-k 1 --prod-deg 2 --prod-k-hi 2 --prod-deg-hi 3 --prod-fill-nl 2
```

# 10 Known gaps

- **The statistical readout does not exist.** The exact-span ridge measure saturates at $H = 0.5$ for any $k \geq 1$, so it cannot see what the term count buys. Every $k$ decision is currently made against a measure blind to precisely that axis. Highest-priority instrument.
- **Band wires are body-static**—never written between the ports, heavily read, hence trivially identifiable. That is what a restriction adversary needs to condition on one. The mirror fill does not fix it; a rolling band would.
- **Static distinguishers are not instrumented at all.** Every heatmap here is execution-based. Gate-statistics along the circuit are unexamined, and the fold-order fix of §4 was found by reading code, not by a measurement that would have caught it.
- **The degree-4 tower** needs scratch decomposition of width-8 fragments.
- **The DB match-rate cost** of width-6 gates is predicted by the $(m, k, c)$ grid law but never measured directly.